

Contents

SYSTEM DESIGN CASE STUDY	2
Introduction & Executive Context	3
Executive Summary	3
The Problem Statement	3
Target State Architecture & Structural Design.....	4
Comprehensive Solution Summary	6
End-End Customer Journey.....	10
Transaction Data-Flows.....	15
Security Architecture	20
Disaster Recovery (DR) Plan	22
Architecture Tradeoffs	24
Architecture Decision Records	25
AWS Well-Architected Framework Review.....	28
Architecture Risks	30
Architectural Recommendations.....	33
Boundary Conditions for Reference Architecture.....	34
Appendix A: Non-Functional Requirements (NFR) Matrix	37
Appendix B: Cost Optimization & Attribution Matrix	40
Appendix C: Enterprise Security Risk Register	43
Appendix D: Enterprise RACI Matrix	46
About the Author.....	47

SYSTEM DESIGN CASE STUDY

Banking & Capital Markets GenAI Architecture on AWS

Prepared by:

Amit Kulkarni

Introduction & Executive Context

This architecture defines a production-ready, highly secure, and compliant cloud blueprint tailored for Capital Markets and Modern Banking Operations. The system transitions traditional financial institutions from passive data storage to an AI-driven, event-based ecosystem. By natively integrating structured banking ledgers, unstructured market feeds, and real-time transaction streams, the platform enables continuous data enrichment. This curated pipeline directly fuels localized Generative AI models to deliver automated fraud detection, automated credit risk profiling, real-time market sentiment analysis, and intelligent client-facing automation.

Executive Summary

This architecture defines a production-ready cloud blueprint tailored for Capital Markets and Modern Banking Operations. The system transitions traditional financial institutions from passive data storage to an AI-driven, event-based ecosystem. By natively integrating structured banking ledgers, unstructured market feeds, and real-time transaction streams, the platform enables continuous data enrichment. This curated pipeline directly fuels localized Generative AI models to deliver automated fraud detection, real-time risk profiling, and intelligent financial reporting.

The Problem Statement

1. Processing Delays and Batch Bottlenecks

- **Overnight Processing Windows:** Legacy core banking systems rely heavily on overnight batch processing, which prevents instant business visibility.
- **Delayed Fraud Detection:** Reactive analysis means fraudulent transactions are often flagged hours or days after they occur.
- **Stale Risk Assessments:** Credit and liquidity risks are calculated using historical snapshots rather than real-time market positions.

2. Severe Data Fragmentation and Silos

- **Disparate Storage Systems:** Critical financial records are trapped across isolated relational databases and transactional logs.
- **Incompatible Formats:** Internal ledger data does not inherently align with unstructured, external third-party market feeds.
- **Complex Ingestion Chains:** Engineering teams spend excessive time building manual point-to-point connectors for new vendors.

3. AI Adoption and Security Barriers

- **Data Leakage Risks:** Passing sensitive financial metrics through public GenAI models violates consumer privacy laws.
- **Lack of Model Grounding:** Generic artificial intelligence engines lack direct, secure access to real-time internal databases.
- **Infrastructure Cost Scarcity:** Provisioning dedicated on-premise hardware for deep learning models demands high upfront capital.

4. Regulatory and Compliance Strain

- **Audit Trail Gaps:** Documenting data lineage across complex, fragmented processing environments is difficult to prove to auditors.
- **Access Control Vulnerabilities:** Perimeter-only security models lack the fine-grained restrictions needed to protect sensitive accounts.
- **Strict Global Frameworks:** Maintaining continuous compliance with mandates like PCI-DSS and Basel III creates heavy operational overhead.

Target State Architecture & Structural Design

System Architecture Overview

The target design establishes a highly available, multi-tier enterprise architecture engineered to connect multi-channel digital user frontends securely to an integrated *AWS data lakehouse* and generative AI ecosystem. The system handles external data ingestion through global content distribution networks and secure transit protocols. All workloads—ranging from transactional banking integration and third-party data streams to trained generative AI models—are hosted across managed serverless components and microservice environments inside a controlled *Virtual Private Cloud (VPC)*.

Core Functional Processing Tiers

1. Global Ingress, Edge Security, & Multi-Channel UI/UX

The application boundary interfaces directly with diverse customer touchpoints, including Mobile, Tablets, Web, and Smart Watch frontends. Global routing, traffic balancing, and edge domain processing are handled via *AWS Route 53* and *AWS CloudFront*. Frontend application deployment, hosting, and static content distribution are accelerated through *AWS Amplify*, which references an asset hosting *AWS S3* Bucket. Dynamic application interaction, client-side requests, and operational control configurations are

explicitly isolated and parsed via *AWS API Gateway* before hitting interior compute environments.

2. Data Ingestion & Lakehouse Integration Pipelines

The platform ingest infrastructure handles structural database synchronizations and multi-format data extraction through three segregated vectors:

- **Bank Data Ingestion:** Core Banking Databases feed real-time transactions directly into a relational backend using *AWS Database Migration Service* (AWS DMS) and *AWS RDS*. High-velocity, real-time transaction streams are captured concurrently via *AWS Kinesis Firehose* to hit an *AWS S3* (Raw Data) lakehouse tier.
- **Third-Party Data Ingestion:** External Market Data and specialized data provider assets flow through *AWS AppFlow* and *AWS Data Exchange*, settling directly into the *AWS S3* Raw Data repository.
- **Lakehouse Processing Engine:** *AWS Glue* cataloging engines and extracttransform-load (ETL) routines clean, structure, and partition the raw database arrays. This outputs clean, compressed schemas into a high-utility *AWS S3* (Curated Data) storage plane, augmented by *AWS EMR* for large-scale distributed analytics.

3. Core Compute, Generative AI Inferences, & Downstream Analytics

Business execution logic, contextual search mappings, and intelligent modeling capabilities run inside managed compute microservices:

- **Serverless Execution Layer:** *AWS Lambda* ingests structural events from the *API Gateway* to route transactional configurations, manage application workflows, and update state metrics inside high-speed *AWS DynamoDB* NoSQL tables.
- **Generative AI Execution Ring:** API pathways interface with *AWS SageMaker* and specialized Trained GenAI Models to serve contextual inferences and ground model pipelines.
- **Enterprise Analytics & Business Intelligence:** Analytical operations extract curated vectors across *AWS Redshift* and serverless *AWS Athena* environments. These platforms aggregate data directly into unified reporting views and executive dashboards using *AWS QuickSight*.

- **Asynchronous Orchestration & Notifications:** Long-running analytical pipelines and batch calculations are coordinated via *AWS Step Functions*, triggering realtime alerts and operation push messages back to the UI/UX frontends via *AWS SNS*.

4. Security, Compliance, & Telemetry Scopes

The surrounding cloud framework is continuously protected by two decoupled operational control towers that run alongside the primary processing pipelines:

- **Authentication & Encryption Group:** Access controls map tightly across the entire architecture. Identity management is centralized within *AWS Cognito*, enforcing federated access and valid token scopes. Fine-grained service-to-service authorization boundaries are configured via *AWS Identity and Access Management (IAM)* least-privilege policies. Cryptographic encryption loops execute dynamically at rest and in transit via *AWS Key Management Service (KMS)* Customer Managed Keys.
- **Audit & Monitoring Group:** Complete system infrastructure state changes, data mutations, and network API trails are permanently logged inside *AWS CloudTrail*. Application errors, database execution metrics, and latency bottlenecks are captured via *AWS CloudWatch* to deliver end-to-end, proactive observability over the computing matrix.

Comprehensive Solution Summary

This production-grade, highly available enterprise architecture delivers an eventdriven, multi-tier cloud topology engineered to securely connect multi-channel consumer interfaces to a robust *AWS data lakehouse* and generative AI ecosystem. Designed to mitigate performance bottlenecks, unify data silos, control cost volatility, and eliminate security exposures inherent in unmanaged AI deployments, this solution leverages *AWS serverless* compute, decoupled analytical pipelines, and automated zero-trust perimeter defense rings. The architecture supports simultaneous high-velocity transactional workloads (such as core banking updates and third-party data streams) alongside advanced machine learning inferences, ensuring operational consistency under volatile workloads.

1. Global Ingress, Edge Security, and Multichannel UI/UX Delivery

The external user interface boundary supports a highly distributed client ecosystem encompassing Mobile applications, Tablet interfaces, standard Web browsers, and Smart Watch touchpoints.

- **Global Domain Management:** Public connection routines terminate at the network boundary using *AWS Route 53*, executing latency-based routing metrics to direct user sessions to the optimal *AWS global backbone* entry node.
- **Edge Delivery and Optimization:** Frontend web shells, static web assets, and edge cached elements are delivered via *AWS CloudFront* to ensure low-latency user interfaces. Frontend integration, runtime orchestration, and edge deployment automation are managed directly via *AWS Amplify*, referencing a secure asset-hosting *AWS S3* Bucket.
- **Perimeter Traffic Sanitation:** Public traffic passing through the *CloudFront* edge is subjected to real-time payload filtering via *AWS WAF* (Web Application Firewall) to drop OWASP Top 10 vulnerabilities, drop known bot patterns, and eliminate adversarial injection payloads. Volumetric infrastructure DDoS defense is sustained at the edge using *AWS Shield Advanced*.
- **API Demarcation and Transit Control:** Dynamic transactional operations (*/api/**) and persistent notifications are mapped to *AWS API Gateway*. The gateway enforces precise rate limiting and client-side query throttling thresholds to mitigate *Denial of Service* (DoS) attacks. *API Gateway* decouples backend processing times from client interaction states, using short-lived tokens to enforce strict tenant isolation before payloads enter the internal *Virtual Private Cloud* (VPC).

2. Cross-Tier Network Segmentation and Microservices Orchestration

The interior application landscape is hosted within an explicitly segmented *AWS Virtual Private Cloud* (VPC) distributed across multiple resilient *Availability Zones* (AZs) to prevent single points of failure.

- **Air-Gapped Network Topology:** The *VPC* is strictly structured into Public *Subnets* (hosting managed *NAT Gateways* for outbound service patches), Private *Compute Subnets* (housing business orchestration pipelines), and Isolated *Data Subnets* (locking down persistent state engines). Transit across these layers avoids the public internet entirely, utilizing *VPC Interface Endpoints* (*AWS PrivateLink*) and regional *Gateway Endpoints*.

Inter-subnet isolation is strictly maintained via stateless *Network Access Control Lists (NACLs)* and *stateful Security Groups* enforcing port-to-port least-privilege pathways.

- **Serverless Logic execution:** Dynamic transaction routing and state mutations are orchestrated by *AWS Lambda* functions. These stateless compute nodes scale dynamically from zero to handle unpredictable concurrency spikes while preventing the idle compute cost drain common in legacy server architectures.
- **Transactional State Storage:** High-velocity user profile data, session context records, and application telemetry metrics write instantly to *AWS DynamoDB* NoSQL tables. *DynamoDB* handles sub-10 millisecond lookups and features built-in multi-region active-active synchronization alongside custom Time-to-Live (TTL) timestamps to automatically purge transient data rows.

3. Data Lakehouse Ingestion and Analytical Transformation Pipelines

The ingestion subsystem handles structured business data, real-time transaction events, and external third-party feeds through three decoupled operational pipelines, converging into a centralized corporate data lakehouse.

- **Bank Data Sync Ingestion:** Inbound production datasets and relational schemas from Core Banking Databases are synchronized continuously via *AWS Database Migration Service (AWS DMS)* directly into low-latency *AWS RDS* instances. Simultaneously, high-velocity real-time transactional streams are handled by *AWS Kinesis Data Firehose*, which groups, buffers, and compresses streaming payloads before dumping raw objects into an *AWS S3 (Raw Data)* bucket tier.
- **Third-Party Data Acquisition:** External financial data streams, market indexes, and specialized data provider structures are ingested securely through pre-built integrations managed by *AWS AppFlow* and *AWS Data Exchange*, routing straight to the *S3 Raw Data* perimeter.
- **Data Lakehouse Transformation Engine:** The analytical foundation centers on a tiered *AWS S3 lakehouse* topology. *AWS Glue* ETL engines execute automated schema scanning, build data catalogs, and initiate *Spark* transformation jobs to parse disjointed raw data, strip anomalies, and write highly compressed, column-oriented *Apache Parquet* formats into an *AWS S3 (Curated Data)* bucket. High-scale distributed big data

workloads and heavy file transformations are augmented using *AWS EMR* clusters.

- **Enterprise Analytics and Ad-hoc Lookups:** Curated business matrices are loaded into an *AWS Redshift* data warehouse cluster for complex multidimensional analysis. Simultaneously, serverless ad-hoc data analysis is executed directly over the *S3* Curated bucket using *AWS Athena*. Reporting layers across *Redshift* and *Athena* consolidate metadata footprints into rich visualizations and operational dashboards via *AWS QuickSight*.

4. **Generative AI Core, Workflow Orchestration, and Notification Ring**

Artificial intelligence features, machine learning workflows, and model evaluations run outside the direct transactional user path, using stateful coordination loops to safeguard output quality.

- **Machine Learning & Model Execution:** Advanced machine learning operations and inference workloads interface directly with *AWS SageMaker* alongside specialized Trained Generative AI Models. The orchestrator calls these endpoints to serve dynamic responses, text extractions, and cognitive tasks.
- **Distributed Logic Orchestration:** Long-running analytics routines, model execution pipelines, and cross-service compute paths are choreographed via *AWS Step Functions* state machines. *Step Functions* track the processing state of multi-step distributed logic, preventing *Lambda* executions from timing out while awaiting external analytical completions.
- **Event Notifications and Client Alerts:** As soon as an analytical milestone or model generation loop is finalized, the pipeline publishes an event to *AWS SNS* (Simple Notification Service). *SNS* fanout rules process the message to push real-time alerts and in-app notifications back down to the UI/UX frontend interfaces.

5. **Zero-Trust Access Control, Data Encryption, and Governance Operations**

Comprehensive compliance frameworks and continuous data loss prevention loops execute globally across the entire active infrastructure matrix.

- **Identity and Access Management:** Identity federation and customer access portals are centralized through *AWS Cognito*, which validates client token scopes. Granular service-to-service execution permissions and

runtime boundaries are configured through explicit *AWS Identity and Access Management* (IAM) role declarations enforcing strict leastprivilege constraints.

- **Global Cryptographic Ring:** Complete data isolation is mandated across the system. All information assets—encompassing active *RDS* instances, *S3* raw and curated buckets, *DynamoDB* records, and *Redshift* clusters—enforce *AES-256* standard encryption at rest using *AWS Key Management Service* (KMS) *Customer Managed Keys* (CMKs) featuring automated key rotation schedules. Data-in-transit pipelines explicitly mandate *TLS 1.3* encryption parameters.
- **System Telemetry and Compliance Visibility:** Every infrastructure state alteration, cloud database update, and service-to-service API call is recorded to an immutable ledger via *AWS CloudTrail*. Application performance analytics, connection states, and exception errors are aggregated continuously within *AWS CloudWatch* to provide proactive observability and alerting.

End-End Customer Journey

Phase 1: Global Edge Connection and Ingress Perimeter Defense

1. Session Initialization and Domain Resolution

- **User Action:** An enterprise client opens the corporate application on their mobile device or web browser and initiates a connection request.
- **Technical Workflow:** The client browser dispatches a secure DNS lookup request. *AWS Route 53* processes the incoming query using latency-based routing policies to resolve the domain to the geographically optimal *AWS CloudFront* edge location. CloudFront initiates the secure TCP handshake and establishes an encrypted *TLS 1.3* connection channel.

2. Edge Content Delivery and Static Shell Loading

- **User Action:** The client interface instantly loads the initial application shell and login layout components.
- **Technical Workflow:** *AWS Amplify* manages the static application distribution, retrieving pre-built layout assets from a secure *AWS S3* Asset Bucket and caching them at the global CloudFront edge nodes. This allows

the client interface to render without consuming central backend server resources.

3. Inline Security Inspection at the Network Boundary

- **User Action:** The user inputs their corporate credentials to authenticate their session.
- **Technical Workflow:** Before the credentials reach any internal compute layers, *AWS WAF V2* intercepts the raw HTTP payload directly at the CloudFront ingress boundary. The WAF engine performs deep packet inspection against OWASP Top 10 lists, cross-site scripting (XSS) vectors, and credential-stuffing signatures. Concurrently, *AWS Shield Advanced* analyzes packet velocity to mitigate volumetric DDoS anomalies.

4. Identity Authentication and Gateway Ingress Management

- **User Action:** The authentication transaction is approved, and the application loads the user's secure control panel.
- **Technical Workflow:** The sanitized payload routes through the public internet boundary into *AWS API Gateway*. API Gateway calls a custom serverless token authorizer, which validates the authentication request against an *AWS Cognito* User Pool. *Cognito* verifies the user's federated claims, signs a short-lived *JSON Web Token (JWT)*, and injects identity claims (such as user ID, organizational tenant ID, and security role parameters) directly into the API routing context map.

Phase 2: Private Compute Orchestration and Data Ingestion Processing

5. Air-Gapped Transit into Internal Corporate Networks

- **User Action:** The user requests a synchronized portfolio evaluation report, pulling live core banking transactions and third-party financial indexes.
- **Technical Workflow:** *API Gateway* maps the user's request metadata and passes it across the air-gapped network boundary using *AWS VPC Interface Endpoints (AWS PrivateLink)*. The transaction lands securely on an *Elastic Network Interface (ENI)* assigned to an *AWS Lambda* orchestrator function situated inside a *Private Compute Subnet*. Strict stateful *Security Groups* block all ingress traffic except for designated gateway interface communication ports.

6. Transaction Synchronization and Live Storage Updates

- **User Action:** The system updates the user's profile view with the latest records from core banking databases.
- **Technical Workflow:** Behind the scenes, *AWS Database Migration Service* (AWS DMS) continuously captures change-data (CDC) matrices from Core Banking Databases and synchronizes them directly into secure *AWS RDS* instances. Simultaneously, live high-velocity financial transactions are streamed through *AWS Kinesis Data Firehose*, which groups, buffers, and dumps raw streaming transaction logs directly into an *AWS S3* (Raw Data) lakehouse repository.

7. Analytical Aggregation and Context Extraction

- **User Action:** The system prepares historical transaction summaries for the user interface.
- **Technical Workflow:** Automated *AWS Glue* ETL workers scan the raw *S3* bucket files, identify metadata structures, catalog schemas, and initiate a distributed Spark processing transformation. This pipeline strips out formatting anomalies, structures text layers, and saves highly compressed, column-oriented *Apache Parquet* blocks into an *AWS S3* (Curated Data) storage plane, augmented by *AWS EMR* for large-scale distributed analytics.

Phase 3: Generative AI Orchestration and Verification Loop

8. Intention Classification and Context Preparation

- **User Action:** The user opens the generative assistant pane and submits a complex query: "Summarize my last 30 days of transactional spending and contrast it with current third-party market trend indexes."
- **Technical Workflow:** The *AWS Lambda* orchestrator parses the prompt text structure using an internal *LangChain/LlamaIndex* framework layer. *Lambda* checks the query's complexity parameter. If it detects a simple extraction or classification task, it scales down routing pathways to a lightweight model tier (such as *Claude Haiku*). For this intensive analytical prompt, it initializes the pipeline to call a high-tier reasoning model (such as *Claude 3.5 Sonnet*) hosted inside *AWS Bedrock* or managed *AWS SageMaker* environments.

9. RAG Grounding and Isolated Vector Search Execution

- **User Action:** The generative assistant begins evaluating the query against the user's personal banking files without data leakage.
- **Technical Workflow:** The *Lambda* orchestrator routes a search payload through private VPC pathways to look up facts. It queries Trained GenAI Models or an AWS S3 Curated Data data pool to gather the necessary context. To ensure absolute data isolation, *Lambda* injects a strict metadata boolean into the query context: WHERE tenant_id == cognito_injected_tenant_id. This prevents cross-tenant data bleed, returning only the user's authorized transaction histories and market records to the compute thread.

10. Streaming Token Inference Delivery

- **User Action:** The user sees the generative assistant's response stream onto their device screen in real time.
- **Technical Workflow:** The *Lambda* orchestrator compiles the final grounded prompt layout—comprising system instructions, data lake context chunks, and the user's raw question—and sends it to AWS *Bedrock*. *Bedrock* triggers model inference processing with response streaming enabled. Token fragments are pushed out immediately as they generate, traveling back through the open *API Gateway* connection to the user frontend interface to minimize perceived latency.

Phase 4: Quality Validation, State Sync, and Audit Logging

11. Output Quality Inspection and Content Filtering

- **User Action:** The final response text block finishes loading cleanly on the user interface display.
- **Technical Workflow:** As the model stream finalizes, the compiled output block is passed to internal validation modules running the *Ragas* and *TruLens* software evaluation frameworks. The logic calculates contextual faithfulness and reference relevance scores. Simultaneously, *Bedrock* Guardrails applies final output validation passes to screen for toxic generation anomalies and enforce strict PII masking constraints.

12. Transactional Session State Commit

- **User Action:** The user closes the assistant pane, knowing their interactive session history will be saved for their next login.
- **Technical Workflow:** The *Lambda* orchestrator compiles a session audit matrix containing the masked user prompt, context linkage metrics, and validated response text. It executes a low-latency PutItem write command to an *AWS DynamoDB* Global Table. *DynamoDB* records the transactional data row across multi-region active-active clusters in under 10 milliseconds and applies a custom Time-to-Live (TTL) epoch attribute to manage automated data pruning.

13. Asynchronous Analytics and Infrastructure Governance Logging

- **User Action:** The active customer session terminates safely.
- **Technical Workflow:** To keep the direct application path clear, the *Lambda* orchestrator pushes telemetry tracks and infrastructure execution trails to an *AWS SQS* FIFO Queue. The queue decouples the front-end user experience from background audit storage pipelines, triggering auxiliary *Lambda* threads that write metrics to long-term *AWS S3* Storage Buckets encrypted via *AWS KMS Customer Managed Keys* (CMKs). Business analysts run trend calculations over this logging tier via *AWS Athena* and *AWS Redshift*, compiling system metrics into unified dashboards using *AWS QuickSight*.

Phase 5: Continuous Operational Compliance and Threat Discovery

14. Real-time Infrastructure Monitoring and Data Loss Protection

- **User Action:** The customer platform operates continuously under strict financial compliance standards.
- **Technical Workflow:** While user transactions execute, *AWS CloudTrail* immutably registers every network API execution trail. *AWS Config* continuously tracks resource properties, scanning security groups and *IAM* profiles against reference architectures to flag exceptions. Concurrently, *AWS Macie* uses machine learning loops to scan the *AWS S3* raw and curated analytics storage tiers, identifying and alerting on any accidentally exposed PII data blocks to maintain continuous *SOC2* and *HIPAA* audit readiness.

Transaction Data-Flows

This transaction flow guide traces the exact sequential transaction flow for an enterprise request moving through the architecture:

Phase 1: Edge Ingress & Perimeter Security Flow

Step 1.1: DNS Resolution & Content Delivery Network Ingress

Protocol & Ingress Boundary: HTTPS (Port 443) / WSS (Port 443) via *AWS Route 53*.

System Processing:

The multi-channel digital client UI (Mobile, Web, Tablet, Watch) dispatches an upstream DNS lookup request. *AWS Route 53* processes the incoming query using latency-based routing policies to return the canonical name (CNAME) pointing to the geographically optimal *AWS CloudFront* Edge location.

Technical Execution: *AWS CloudFront* initiates a TCP three-way handshake and terminates the external *TLS 1.3* session using an *AWS Certificate Manager* (ACM) wildcard certificate. Static frontend application assets are delivered via Edge cache hits from an asset-hosting *AWS S3* Bucket, while dynamic application request pathways (*/api/**) are marked for secure origin-forwarding.

Step 1.2: Edge Security Inspection & Packet Mitigation

Component Intersection: *AWS WAF V2* and *AWS Shield Advanced*.

System Processing: *AWS CloudFront* streams the raw HTTP request header and payload byte array to an inline *AWS WAF V2* WebACL instance for real-time traffic sanitation.

Technical Execution: The *WAF* engine runs deep packet inspection checks against managed rule groups (*OWASP* Top 10 vulnerabilities, SQL injections, Cross-Site Scripting, and known malicious GenAI prompt injection signatures). Concurrently, *AWS Shield Advanced* executes inline heuristic rate tracking against the originating IP. If traffic velocity or payload signatures violate safety thresholds, *WAF* intercepts the transit flow at the edge and returns an HTTP 403 Forbidden or HTTP 429 Too Many Requests code, preventing the packet from touching the core network.

Step 1.3: Gateway Routing, Authentication & Context Injection

Component Intersection: *AWS API Gateway* and *AWS Cognito*.

System Processing: Validated HTTP payloads flow over the *AWS global network backbone* into a regional *AWS API Gateway* instance acting as the air-gapped system perimeter manager.

Technical Execution:

API Gateway intercepts the incoming payload and triggers a synchronous HTTP POST call to an isolated serverless Custom *Lambda* Authorizer.

The Authorizer extracts the *Bearer JWT* token from the Authorization header and validates its cryptographic signature against the *AWS Cognito* User Pool JSON Web Key Set (JWKS). *Cognito* decodes the user's federated identity claims and extracts the session metadata context (e.g., *user_id*, *tenant_id*, and *security_role*). The authorizer maps these parameters into an *IAM* Policy document and injects the client context parameters directly into the *\$context.authorizer* JSON payload map. *API Gateway* then applies token-bucket throttling metrics before passing the request down-funnel.

Phase 2: Decoupled Compute & Lakehouse Ingestion Flow

Step 2.1: Private Network Transit & Microservice Invocations

Protocol & Ingress Boundary: *VPC Interface Endpoints (AWS PrivateLink)*.

System Processing: *API Gateway* translates the authenticated client request into an integration proxy payload and dispatches it securely across the VPC perimeter boundary.

Technical Execution: The integration proxy payload travels through an *Elastic Network Interface (ENI)* allocated to an *AWS Lambda* orchestrator function situated inside a *Private Compute Subnet*. Strict network isolation is enforced at the interface tier: stateful Security Groups block all ingress traffic except for designated gateway interface communication ports, while stateless *Network Access Control Lists (NACLs)* restrict traffic movements to port-specific microperimeters.

Step 2.2: Live Banking Synchronization & Raw Storage Ingestion

Component Intersection: *AWS DMS*, *AWS RDS*, *AWS Kinesis Data Firehose*, and *AWS S3 (Raw Data)*.

System Processing: Real-time production core banking updates and ingestion data streams execute concurrently out of the primary application loop.

Technical Execution: *AWS Database Migration Service (AWS DMS)* continuously intercepts change-data-capture (CDC) metrics from Core Banking Databases and

executes low-latency replication into an *AWS RDS* relational cluster inside the isolated data subnets. Simultaneously, high-velocity real-time transactional streams and external third-party structures (ingested via *AWS AppFlow* and *AWS Data Exchange*) hit an *AWS Kinesis Data Firehose* streaming delivery endpoint. Kinesis buffers, batches, and applies GZIP compression to the incoming transaction byte streams, executing an automated multipart upload into an *AWS S3 (Raw Data) lakehouse* storage bucket.

Step 2.3: Lakehouse Extraction, Cataloging & Transformation

Component Intersection: *AWS Glue*, *AWS S3 (Curated Data)*, and *AWS EMR*.

System Processing: Raw multi-format files and transaction schemas are processed asynchronously into analytical representations.

Technical Execution: An *AWS Glue* crawler reads new raw objects in the *S3* bucket, identifies schema mutations, and writes updated metadata metrics to the central *AWS Glue Data Catalog*. An automated *AWS Glue* ETL job launches a distributed *Apache Spark* processing cluster, or triggers an auto-scaling *AWS EMR* pipeline for heavy batch calculations. The *Spark* engine parses the unstructured text rows, strips out formatting anomalies, partitions the data fields by date and tenant IDs, and outputs highly compressed, column-oriented *Apache Parquet* blocks into the *AWS S3 (Curated Data)* storage plane.

Phase 3: Generative AI Orchestration & RAG Grounding Flow

Step 3.1: Intention Classification & Model Cascading

Component Intersection: *AWS Lambda* (Orchestrator Core running *LangChain/LlamaIndex*).

System Processing: The core *Lambda* orchestrator handles the incoming generative assistant query to compute the most cost-efficient execution pathway.

Technical Execution: The *Lambda* function initializes an internal *LangChain/LlamaIndex* processing framework to evaluate prompt intent and syntactic complexity. If the query matches basic text classification, entity extraction, or structured data formatting layout rules, the logic sets the target endpoint flag to route to a low-cost model tier (such as *Anthropic Claude 3.5 Haiku*). If the prompt requires multi-step evaluation, data comparison, or deep reasoning, the router switches the execution target to a high-tier reasoning

model (such as *Anthropic Claude 3.5 Sonnet*) hosted within *AWS Bedrock* or managed *AWS SageMaker* environments.

Step 3.2: Cross-Tenant Isolated Vector Search & Grounding

Component Intersection: *AWS OpenSearch Serverless / AWS Kendra*.

System Processing: The *RAG* framework queries corporate content databases to isolate contextual facts while maintaining multi-tenant safety.

Technical Execution:

The *Lambda* orchestrator generates an text query vector embedding and fires an internal API lookup request across private *VPC Gateway Endpoints* to *AWS OpenSearch Serverless* or *AWS Kendra*. To enforce complete database isolation, the search payload appends a strict boolean metadata filter extracted from the *Cognito* context: `WHERE tenant_id == $context.authorizer.tenant_id`. The search engine runs a K-Nearest Neighbors (KNN) match inside the tenant's isolated data namespace, isolates the top relevant source text blocks (Top-K), and returns the clear text chunks back to the *Lambda* orchestrator thread.

Step 3.3: Token Streaming Generation & Perimeter Outflow

Component Intersection: *AWS Bedrock* (Inference Gateway Runtime) and *AWS Bedrock Guardrails*.

System Processing: The system synthesizes the final grounded prompt layout and initiates the token streaming generation loop.

Technical Execution: *Lambda* compiles the finalized template string format: `[System Core Prompt] + [OpenSearch Isolated Context Chunks] + [User Assistant Query]`. The orchestrator passes the string payload to the *Bedrock* `InvokeModelWithResponseStream` API, applying strict input validation rules via *AWS Bedrock Guardrails* and *NeMo Guardrails* to filter prompt injections. The model processes the payload and returns token fragments chunk-by-chunk over a stateful streaming connection. *API Gateway* receives these fragments and pushes them over the active *WebSocket* or *HTTP/2* transport connection directly back to the client interface browser.

Phase 4: Output Validation, State Sync, & Analytics Logging Flow

Step 4.1: Inline Output Safety & Quality Inspection

Component Intersection: *Ragas / TruLens* Frameworks and *AWS Bedrock Guardrails*.

System Processing: The compiled response text undergoes automated compliance and hallucination checks prior to UI acceptance.

Technical Execution: As the response stream finalizes, the compiled text block is passed to internal validation modules running the *Ragas* or *TruLens* software evaluation frameworks. The logic calculates contextual faithfulness and reference relevance scores. Simultaneously, *Bedrock* Guardrails applies final output validation passes to screen for toxic generation anomalies and enforce strict PII masking constraints.

Step 4.2: Session State Sync Commit

Component Intersection: *AWS DynamoDB* Global Tables.

System Processing: The orchestrator commits the transactional session history metadata to the persistence database tier.

Technical Execution: *Lambda* maps out a structured PutItem transaction payload containing the `session_id`, `timestamp`, `masked_prompt`, and `validated_response`. It dispatches this payload over a private endpoint to an *AWS DynamoDB* Global Table. *DynamoDB* processes the write and completes sub-10 millisecond active-active replication to configured secondary disaster recovery *AWS regions*, applying an automated Time-to-Live (TTL) epoch attribute to manage automated data pruning.

Step 4.3: Asynchronous Ingestion & Analytics Data Warehouse Load

Component Intersection: *AWS SQS* FIFO Queue, *AWS Redshift* Serverless, *AWS Athena*, and *AWS QuickSight*.

System Processing: Ingress logs, telemetry trails, and system state histories are offloaded asynchronously to the analytics data warehouse.

Technical Execution: The primary *Lambda* orchestrator pushes a tracking payload to an *AWS SQS* FIFO Queue, ensuring message delivery order is maintained. The *SQS* queue triggers background batch ingestion *Lambda* workers, completely decoupling the user interface from long data parsing routines. These workers stream logging text objects into an *AWS S3* Curated Data Bucket wrapped in *AES-256* encryption via *AWS KMS Customer Managed Keys* (CMKs). An automated *AWS Step Functions* state machine orchestrates the copy command to load partitioned *Parquet* datasets directly into *AWS Redshift* Serverless or queries them ad-hoc via *AWS Athena*. *AWS QuickSight* business intelligence engines pull these tables to render executive dashboards. Real-time

operational alerts or push messages are handled by *AWS SNS* to signal frontend clients upon transaction completion.

Phase 5: Security Control & Governance Audit Flow

Step 5.1: Automated Compliance Scans & Threat Detections

Component Intersection: *AWS CloudTrail*, *AWS Config*, and *AWS Macie*.

System Processing: Security control systems continuously scan network mutations and data classifications to enforce governance rules.

Technical Execution: Every *IAM* service role execution, database query mutation, and model invocation is permanently written to an unalterable *AWS CloudTrail* ledger for compliance monitoring. *AWS Config* continuously tracks resource property snapshots, immediately generating security hub alert tickets if *subnets*, *IAM* profiles, or *KMS* key rotation rules fall out of compliance boundaries. *AWS Macie* runs automated machine learning loops across *S3* analytics buckets to scan file objects for accidentally exposed PII data or unencrypted text columns, closing the loop on continuous compliance management.

Security Architecture

This architecture uses a zero-trust model to protect sensitive financial records and comply with global regulations (such as *PCI-DSS* and *SOC 2*). Security controls are built directly into every layer of the data and computing infrastructure, Architecture encompasses AI Model Guardrails & Data Privacy (protecting *AWS SageMaker* data leaks) and Infrastructure Observability & Operational Resilience

1. Data Encryption & Key Management

- **Enforced Storage Cryptography:** All data within the Raw and Curated *AWS S3 lakes*, *AWS RDS*, and *AWS DynamoDB* is encrypted at rest using *AES256*.
- **Centralized Key Control:** *AWS KMS* manages and rotates the customermanaged keys (CMKs) used across all ingestion and analytical services.

Strict Transit Protection: *TLS 1.3* encryption protects all data moving between edge clients, *AWS API Gateway*, *serverless functions*, and backend databases.

- **Hardware-Backed Security:** Cryptographic keys are isolated inside dedicated FIPS 140-3 Level 3 validated hardware security modules.

2. Fine-Grained Access Control & Governance

- **Automated Data Cataloging:** *AWS Glue Data Catalog* discovers, catalogs, and tags sensitive financial assets and personally identifiable information (PII).
- **Column and Row Security:** Detailed access policies limit database visibility within *AWS Athena* and *AWS Redshift* based on user role.
- **Continuous Integrity Audits:** *AWS CloudTrail* logs and indexes every API transaction to provide an unalterable trail for regulatory compliance.
- **Storage Lifecycle Automation:** *AWS S3 Object Lock* policies prevent data tampering, helping the platform meet strict regulatory record-retention mandates.

3. Identity & Least Privilege Access Management

- **Decoupled User Identity:** *AWS Cognito* handles consumer authentication, registration tokens, and multi-factor authentication (MFA) at the application edge.
- **Context-Driven Policy Restrictions:** Role-based *AWS IAM* policies enforce the principle of least privilege for every application and system account.
- **Temporary Session Security:** System workloads use *AWS STS* to request short-lived credentials, which eliminates the security risk of hardcoded API keys.
- **Cross-Service Execution Boundaries:** Strict execution roles isolate serverless *AWS Lambda* functions, ensuring they can only access specified target systems.

4. Boundary Protection & Network Security

- **Isolated Cloud Networking:** Compute and database resources run within private subnets across multiple availability zones inside an *AWS VPC*.
- **Edge Attack Prevention:** *AWS CloudFront* combines with *AWS WAF* to block common web application attacks, bad bots, and DDoS disruptions.
- **Private Network Ingestion:** Third-party data feeds pass through secure *AWS AppFlow* endpoints, keeping transaction traffic off the public internet.
- **Stateful Access Separation:** Strict *Security Groups* and *Network Access Control Lists (NACLs)* restrict direct access to databases.

5. AI Model Guardrails & Data Privacy

- **Air-Gapped ML Processing:** *AWS SageMaker* training and inference run inside isolated subnets with internet access disabled to prevent data exfiltration.
- **PII Redaction Ingestion:** Streaming *AWS Lambda* triggers automatically mask or strip account numbers and personal details before saving to the *AWS S3 Raw Lake*.
- **Prompt and Response Filtering:** Infrastructure guardrails monitor incoming GenAI tokens to block toxic inputs and prevent confidential model leaks.
- **Model Lineage Auditing:** *AWS SageMaker* ML Lineage tracking automatically maps the exact dataset version used to train every deployed AI model.

6. Infrastructure Observability & Operational Resilience

- **Real-Time Log Ingestion:** *AWS CloudWatch* aggregates logs, metrics, and application traces from all microservices to alert on system runtime faults.
 - **VPC Flow Log Analysis:** Network telemetry is constantly monitored to flag and isolate unexpected traffic patterns or unrecognized API requests.
 - **Multi-AZ Fault Isolation:** Ingestion streams and database layers run across multiple *AWS Availability Zones* to guarantee high availability during hardware failures.
- Proactive Alert Dissemination:** Downstream system health alerts pass through *AWS SNS* to notify engineering teams via automated communication channels.

Disaster Recovery (DR) Plan

This architecture uses a Multi-Region Backup and Hot-Standby strategy to maintain operational resilience and business continuity. It meets strict financial service targets:

- **Recovery Point Objective (RPO)** under 1 minute
- **Recovery Time Objective (RTO)** under 15 minutes.

1. Multi-Region Data Replication Strategy

- **Cross-Region S3 Storage:** The Raw and Curated *AWS S3 lakes* use versioned, automated cross-region replication to duplicate data to a secondary DR region.
- **Global Relational Database:** *AWS RDS* runs a cross-region read replica that can be promoted to a primary database instantly if a regional failure occurs.

- **NoSQL Database Replication:** *AWS DynamoDB* Global Tables provide fully managed, active-active multi-region replication to keep application state data synchronized.
- **Immutable Backup Enforcement:** *AWS Backup* enforces WORM (Write Once, Read Many) compliance to protect point-in-time database backups from ransomware or deletion.

2. Automated Network and Failover Routing

- **Active-Passive DNS Failover:** *AWS Route 53* health checks monitor primary region endpoints and automatically reroute client traffic during an outage.
- **Global Dynamic Routing:** *AWS CloudFront* uses origin failover groups to switch its primary path from the main *AWS API Gateway* to the standby DR region without user disruption.
- **Serverless Compute Provisioning:** *AWS Lambda* functions and *AWS API Gateway* configurations are deployed across regions using Infrastructure as Code (IaC) for immediate processing.
- **Stateful Stream Recovery:** *AWS Kinesis* consumers store iteration checkpoints externally, letting processing pipelines pick up exactly where they left off after a failover.

3. ML Model and Core Pipeline Resilience

- **Model Registry Mirroring:** *AWS SageMaker* Model Registry replicates trained weights and metadata to the target backup region for immediate container deployment.
- **Decoupled Job Resubmission:** *AWS Glue* and *AWS EMR* utilize stateless cluster configurations that can spin up and resume analytics jobs using replicated *AWS S3* data.
- **Fallback Rule Configurations:** Client applications run a local rules engine that bypasses AI enrichment if *AWS SageMaker* timeouts occur, keeping core transactions processing.
- **Independent Parameter Syncing:** Model prompt templates and operational parameters are backed up continuously inside region-agnostic parameter stores.

4. Continuous Testing and Validation

- **Chaos Engineering Drills:** Automated simulations inject faults into network paths and database clusters to verify self-healing mechanisms work correctly.

- **Automated Lineage Verification:** Post-failover scripts run validation queries to check data consistency between the primary and recovery regions.
RTO and RPO Auditing: System logs monitor the speed of automated failovers to prove compliance with financial regulators and SLA metrics.
- **Immutable Infrastructure Drills:** Teams periodically tear down and rebuild standby environments using *AWS CloudFormation* to prevent configuration drift.

Architecture Tradeoffs

1. Serverless Compute vs. Provisioned Clusters

- **The Benefit:** Utilizing serverless components like *AWS Lambda*, *AWS Glue*, and *AWS Athena* minimizes infrastructure management overhead and scales down to \$0 when inactive.
- **The Trade-off:** High-concurrency financial events or complex analytics queries can suffer from cold-start latency and volatile, transaction-heavy compute pricing spikes.
- **The Mitigation:** Implement Provisioned Concurrency for time-sensitive *AWS Lambda* functions, utilize *AWS EMR* for consistent, large-scale batch processing, and enforce execution timeouts.

2. Micro-Segmented Zero-Trust Perimeter vs. Network Performance

- **The Benefit:** Routing all data feeds through private *VPC endpoints*, *AWS AppFlow*, inspection boundaries, and multi-layered firewalls removes exposure to public internet attacks.
- **The Trade-off:** Layering *AWS API gateways*, transit proxies, and deep packet analysis introduces unavoidable millisecond latency bottlenecks into the transaction data flow.
- **The Mitigation:** Use *AWS CloudFront* edge acceleration for frontend caching, set up *VPC endpoint* routing tables, and use protocol optimizations like HTTP/3.

3. Air-Gapped Generative AI Models vs. Innovation Velocity

- **The Benefit:** Running *AWS SageMaker* inside strictly isolated, internet-disabled *subnets* prevents data exfiltration, ensuring sensitive corporate metrics never leak to public models.

- **The Trade-off:** Restricting external network access complicates model fine-tuning, blocks direct ingestion of external third-party AI APIs, and increases internal engineering overhead.
- **The Mitigation:** Establish secure, automated container pipelines that scan, approve, and mirror open-source foundational models into an internal registry via an inspected *transit VPC*.

4. Continuous Real-Time Ingestion vs. Storage & Computational Cost

- **The Benefit:** Pushing continuous transaction events through *AWS Kinesis* allows for immediate, sub-second fraud detection and live reporting.
- **The Trade-off:** Keeping computing resources running 24/7 generates massive volumes of small files in the raw data lake, which reduces query performance and drives up infrastructure costs.
- **The Mitigation:** Use *AWS Kinesis Data Firehose* to compress data into efficient formats like *Apache Parquet* and run automated hourly *AWS Glue* consolidation jobs to merge small files.

5. Multi-Region Active-Passive Resiliency vs. Operational Complexity

- **The Benefit:** Running cross-region replication via *AWS S3*, *AWS RDS*, and *AWS DynamoDB* guarantees an RTO under 15 minutes, safeguarding the platform against full regional cloud failures.
- **The Trade-off:** Managing distributed infrastructure creates a high risk of split-brain data conflicts during network partitions, increases database licensing costs, and introduces data drift between regions.
- **The Mitigation:** Enforce deterministic timestamp ordering using *AWS DynamoDB* Global Tables, run automated *AWS CloudFormation* templates to eliminate configuration drift, and use *AWS Route 53* health checks to manage automated failovers.

Architecture Decision Records

1. Centralized Data Lake over Traditional Data Warehousing

Decision: Adopt a multi-tiered *AWS S3 data lake* architecture (Raw/Curated) as the primary storage layer, rather than staging all inbound transactional records inside a relational data warehouse.

Rationale: This separates storage costs from computing costs, supports structured, semi-structured, and unstructured data formats, and scales to

petabyte levels cheaply to serve as the foundation for training *AWS SageMaker* models.

2. Stream-First Ingestion via Kinesis over Batch-Oriented ETL

- **Decision:** Use *AWS Kinesis Data Firehose* to ingest transaction logs continuously, replacing legacy daily or hourly batch-processing schedules.
- **Rationale:** This brings data into the cloud in sub-second times, which is necessary to run real-time fraud detection algorithms, generate instant risk alerts, and provide up-to-the-minute business intelligence dashboards.

3. Change Data Capture (CDC) via AWS DMS for Core Ledgers

- **Decision:** Deploy *AWS Data Migration Service (DMS)* with continuous replication to stream core ledger modifications into *AWS RDS* and the *AWS S3 data lake*.
- **Rationale:** This extracts transactional updates directly from database transaction logs without running heavy SQL queries, avoiding performance drops on production banking applications.

4. Fully Isolated, Air-Gapped VPC Subnets for AI Workloads

- **Decision:** Lock down *AWS SageMaker* training and inference environments inside private *subnets* with internet access disabled via strict security groups and routing tables.
- **Rationale:** This prevents data leakage and ensures proprietary financial data, client portfolio metrics, and internal banking parameters cannot be exfiltrated to public internet endpoints or unauthorized third-party AI APIs.

5. Serverless ETL Pipelines via AWS Glue over Provisioned Compute

- **Decision:** Standardize data transformation, partitioning, and metadata cataloging on serverless *AWS Glue* jobs, avoiding dedicated server clusters.
- **Rationale:** This removes the operational burden of patching and scaling servers, while its pay-per-use billing scales data preparation costs down to zero during off-peak trading hours.

6. Decoupled Edge Security with API Gateway and CloudFront

- **Decision:** Funnel all application and mobile platform requests through an integrated edge layer using *AWS CloudFront*, *AWS WAF*, and *AWS API Gateway*.

- **Rationale:** This blocks distributed denial-of-service (DDoS) attacks, stops bad bots at the network edge, and validates inbound API structures before traffic ever reaches core financial microservices.

7. Global Identity Management via AWS Cognito

- **Decision:** Offload user authentication, token generation, and multi-factor authentication (MFA) workflows entirely to *AWS Cognito*.
- **Rationale:** This separates user authentication logic from backend business code, ensuring the platform securely handles user identities in compliance with global standards like *PCI-DSS*.

8. Serverless Ad-Hoc Analytics via AWS Athena

- **Decision:** Use *AWS Athena* as the primary tool for ad-hoc compliance checks, data discovery, and internal auditing against the curated *AWS S3* data lake.
- **Rationale:** This allows risk teams to query raw files directly using standard SQL without needing to load datasets into a expensive, always-on analytics cluster.

9. Multi-Region Active-Passive Architecture for Core Storage

- **Decision:** Configure cross-region automated replication for *AWS S3*, *AWS RDS*, and *AWS DynamoDB* to back up data into a secondary disaster recovery region.
- **Rationale:** This safeguards operations against complete regional cloud outages, allowing the infrastructure to meet strict banking uptime rules with an RTO under 15 minutes.

10. Managed Customer Keys via AWS KMS with Automatic Rotation

- **Decision:** Secure all system storage with customer-managed cryptographic keys inside *AWS KMS*, combined with automated annual key rotation policies.
- **Rationale:** This gives the organization complete control over data encryption access, isolates encryption tasks within certified hardware security modules, and fulfills strict financial regulatory audit requirements.

AWS Well-Architected Framework Review

An evaluation of this architecture against the design principles of the AWS WellArchitected Framework reveals the following target alignments and gaps:

1. Security Pillar

- **Strengths:**
 - Implements a zero-trust model using micro-segmented VPC environments.
 - Air-gaps *AWS SageMaker* within internet-disabled subnets to prevent financial data leaks.
 - Uses automated encryption at rest and in transit via customermanaged keys in *AWS KMS*
- **Recommendations:**
 - Implement *AWS Secrets Manager* with automated rotation for database credentials to phase out static *IAM* execution roles where possible.
 - Deploy *AWS Macie* to scan the *AWS S3 Raw Lake* automatically for accidental uploads of unmasked PII.

2. Reliability Pillar

- **Strengths:**
 - Uses active-passive cross-region replication for core storage layers (*AWS RDS*, *AWS DynamoDB*, and *AWS S3*).
 - Automates user routing during failures via *AWS Route 53* health checks.
 - Decouples ingestion streams with *AWS Kinesis*, preventing traffic spikes from overwhelming backend applications.
- **Recommendations:**
 - Use *AWS Fault Injection Service* (FIS) to run automated chaos engineering drills on RDS failover paths.
 - Configure dead-letter queues (DLQs) on all processing *AWS Lambda* functions to isolate corrupted or unparseable financial payloads.

3. Cost Optimization Pillar

- **Strengths:**
 - Implements serverless pay-per-use architectures, scaling costs down to zero during market closure hours.
 - Uses a tiered data lake model, avoiding the high costs of keeping raw, unrefined logs inside relational databases.

- **Recommendations:**
 - Apply *AWS S3 Lifecycle* policies to transition raw transactional logs automatically to *AWS S3 Glacier Flexible Retrieval* after 90 days.
 - Use *AWS Compute Optimizer* to identify over-provisioned instance classes running inside the *AWS EMR* or *AWS SageMaker* training clusters.

4. Performance Efficiency Pillar

- **Strengths:**
 - Shifts workloads from legacy batch schedules to sub-second real-time streaming ingestion.
 - Uses serverless compute engines (*AWS Glue*, *AWS Athena*) to scale processing instantly during peak trading hours.
 - Offloads global frontend distribution and caching to *AWS CloudFront* edge nodes.
- **Recommendations:**
 - Tune *AWS Kinesis Data Firehose* buffering hints (e.g., matching size and time targets) to minimize the file count generated in the raw lake.
 - Establish a data-tiering strategy that moves historical ledger data from *AWS Redshift* SSDs to *AWS S3* cold storage

5. Operational Excellence Pillar

- **Strengths:**
 - Uses *AWS CloudTrail* and *AWS CloudWatch* to centralize API audit paths and application performance metrics.
 - Tracks machine learning models through *AWS SageMaker* Lineage, mapping deployments directly back to their source datasets.
- **Recommendations:**
 - Define the entire infrastructure landscape exclusively through reusable Infrastructure as Code (IaC) templates using the *AWS Cloud Development Kit (CDK)*.
 - Build automated dashboards in *AWS CloudWatch* to track *Key Performance Indicators (KPIs)* like end-to-end transaction latency and API error rates.

6. Sustainability Pillar

- **Strengths:**
- Uses shared, multi-tenant serverless resources (*AWS Lambda*, *AWS Athena*, and *AWS Glue*) to maximize hardware utilization and minimize idle carbon emissions.
- **Recommendations:**
- Review ML training schedules to run resource-heavy *AWS SageMaker* optimization jobs in *AWS regions* powered by higher percentages of renewable energy.
- Compress raw financial logs using high-efficiency formats like *Zstandard* before storing them long-term in *AWS S3*.

Architecture Risks

1. Inefficient Small File Storage Patches

- **The Flaw:** Continuous, real-time transaction streaming via *AWS Kinesis Data Firehose* can write thousands of tiny JSON files into the raw storage layer every hour.
- **The Risk:** Analytical queries run via *AWS Athena* will slow down significantly, while metadata query costs and *AWS S3* API call expenses will spike.
- **The Fix:** Configure *AWS Kinesis* buffering limits to at least 128MB or 5 minutes, compress payloads using *Apache Parquet*, and schedule daily *AWS Glue* jobs to merge small files.

2. Live Network Access During AI Fine-Tuning

- **The Flaw:** Leaving internet access enabled on *AWS SageMaker* notebook instances or training jobs while refining financial models.
- **The Risk:** Malicious dependencies, compromised third-party open-source packages, or prompt injection exploits could exfiltrate proprietary financial data to external servers.
- **The Fix:** Explicitly disable internet access on *AWS SageMaker* resources, routing traffic exclusively through private *VPC Endpoints*, and pull verified base models from an internal repository.

3. Missing Real-Time Data Masking Policies

- **The Flaw:** Writing raw multi-channel transaction data directly into the *AWS S3 data lake* before stripping out sensitive customer identifiers.

- **The Risk:** Unmasked personally identifiable information (PII) becomes accessible to data engineers and downstream business analytics workloads, violating strict *PCI-DSS* regulations.
- **The Fix:** Deploy an inline *AWS Lambda* function within *AWS Kinesis Data Firehose* to automatically hash, redact, or mask sensitive fields before writing data to S3.

4. Over-Provisioned Shared Lambda Execution Roles

- **The Flaw:** Assigning a single, broad *AWS IAM* execution role to multiple backend *AWS Lambda* microservices instead of tailoring permissions per function.
- **The Risk:** If one customer-facing API function is compromised, attackers can use its wide permissions to modify database records or access unrelated storage lakes.
- **The Fix:** Build dedicated, least-privilege *IAM* roles for every single *Lambda* function, strictly limiting access to the exact tables or buckets required for its specific task.

5. Uncontrolled Lambda Cold Start Timeouts

- **The Flaw:** Relying entirely on standard on-demand *AWS Lambda* functions to handle synchronous, customer-facing transaction requests through *AWS API Gateway*.
- **The Risk:** Sudden traffic spikes after periods of inactivity trigger cold starts, causing API timeouts and disrupting the client transaction experience.
- **The Fix:** Enable Provisioned Concurrency for core transaction-processing *AWS Lambda* functions to keep a baseline of execution environments warm and ready.

6. Vulnerable Database Credentials Inside Code

- **The Flaw:** Hardcoding database passwords or access tokens inside application code repositories for connections between backend services and *AWS RDS*.
- **The Risk:** Source code leaks or unauthorized internal access expose the core banking ledger to total compromise and data alteration.
- **The Fix:** Store all database credentials in *AWS Secrets Manager*, enforce automated 30-day password rotation, and use *IAM* database authentication to remove static passwords entirely.

7. Missing Circuit Breaker for AI Models

- **The Flaw:** Synchronously blocking core payment and transaction processing while waiting for *AWS SageMaker* models to finish risk and fraud calculations.

- **The Risk:** If the *AWS SageMaker* inference cluster experiences high latency or an outage, the entire core banking transaction pipeline freezes, halting client payments.
- **The Fix:** Implement an asynchronous circuit-breaker pattern; if the AI model fails to respond within a few milliseconds, fall back to a localized, rules-based safety check.

8. Single-AZ Storage Cluster Bottlenecks

- **The Flaw:** Configuring operational databases (*AWS RDS*) or stream processing nodes to run inside a single *AWS Availability Zone* without hotstandbys.
- **The Risk:** A localized data centre hardware failure or power outage will cause immediate data loss and take down the entire financial platform.
- **The Fix:** Convert the *AWS RDS* deployment to *Multi-AZ*, and configure *AWS Kinesis* streams to replicate data natively across multiple availability zones.

9. Blind Spots in API Audit Tracking

- **The Flaw:** Leaving *AWS CloudTrail* data events or *AWS CloudWatch VPC flow logs* disabled to save on infrastructure logging costs.
- **The Risk:** The organization cannot trace or reconstruct data access paths during an internal breach, leaving security teams unable to prove compliance to financial auditors.
- **The Fix:** Turn on *AWS CloudTrail* multi-region logging for management and data events, enforce *AWS S3* server access logging, and store audit records in a separate, immutable security account.

10. Drift in Deployed Model Accuracy

- **The Flaw:** Deploying a high-performance *AWS SageMaker* predictive risk model without an automated pipeline to check for data and model drift over time.
- **The Risk:** As market conditions shift, the static model's predictions become inaccurate, resulting in missed fraudulent behavior or incorrect credit risk assessments.
- **The Fix:** Deploy *AWS SageMaker Model Monitor* to continuously compare real-time inference data against training baselines, triggering alerts and automated retraining cycles when drift is detected.

Architectural Recommendations

1 . Guarding Against Financial PII Leaks

- **Action:** Integrate *AWS Macie* into the data lifecycle.
- **Recommendation:** Configure automated, daily cryptographic scans across both the Raw and Curated *AWS S3* data lakes to discover, alert on, and isolate unmasked credit card numbers or personally identifiable information (PII) before it is read by downstream AI models.

2. Eliminating Static Infrastructure Passwords

- **Action:** Standardize secret ingestion via *AWS Secrets Manager*.
Recommendation: Remove all hardcoded or configuration-file credentials for *AWS RDS* and *AWS Redshift*, implementing automated 30-day password rotation policies and enforcing *IAM*-based database authentication for all microservices.

3. Preserving Immutability for Regulatory Audits

- **Action:** Deploy *S3 Object Lock* in Compliance Mode.
- **Recommendation:** Enforce an unalterable Write-Once-Read-Many (WORM) storage policy on the Curated *AWS S3 Data Lake* for a minimum retention window of seven years to fulfill strict regulatory requirements (such as FINRA and SEC rules).

4. Overcoming Transaction Latency Spikes

- **Action:** Enforce Provisioned Concurrency configurations.
- **Recommendation:** Allocate dedicated, warm execution environments to core banking *AWS Lambda* functions handling critical customer transaction endpoints via *AWS API Gateway*, keeping end-to-end processing times under acceptable millisecond thresholds.

5. Managing Storage Costs via Automated Archival

- **Action:** Implement multi-tier *AWS S3 Lifecycle Policies*.
- **Recommendation:** Move unstructured log data automatically from *S3 Standard* to *S3 Glacier Flexible Retrieval* after 90 days, and configure permanent purging after 365 days to lower data lake storage costs.

6. Safeguarding Payments Against Model Outages

- **Action:** Deploy an Asynchronous Circuit-Breaker Pattern.

- **Recommendation:** Decouple the core *AWS RDS* transactional flow from *AWS SageMaker* inference endpoints by establishing a millisecond timeout that falls back to a rules-based system if the AI model becomes unresponsive.

7. Preventing Model Drift in Volatile Markets

- **Action:** Turn on *AWS SageMaker* Model Monitor.
- **Recommendation:** Set up automated monitoring to compare incoming production transaction payloads against baseline training datasets, alerting risk teams and triggering automated retraining loops when accuracy thresholds drop.

8. Simplifying Infrastructure Scaling and Auditing

- **Action:** Standardize deployments on the *AWS Cloud Development Kit* (CDK).
- **Recommendation:** Replace manual console changes with declarative, componentized Infrastructure as Code (IaC) pipelines, allowing engineering teams to audit network configurations and clone environments deterministically across disaster recovery regions.

9. Isolating Defective Transaction Payloads

- **Action:** Implement Dead-Letter Queues (DLQs) across streaming steps.
- **Recommendation:** Connect *AWS SQS* dead-letter queues to all *AWS Kinesis* processing *Lambda* functions to isolate corrupted data payloads without breaking the processing loop for the rest of the stream.

10. Proactively Simulating Cloud Regional Outages

- **Action:** Integrate *AWS Fault Injection Service* (FIS) into testing environments.
- **Recommendation:** Run automated monthly chaos engineering drills that simulate network cuts and database failures, validating that *AWS Route 53* and active-passive failover routes switch automatically within target disaster recovery windows.

Boundary Conditions for Reference Architecture

1 Ingestion Volume and Throughput Limits

- **The Issue:** Sudden, massive market movements or high-frequency trading spikes can overwhelm raw data streams.

The Condition: The ingestion pipeline is built to handle a baseline of up to 10,000 transactions per second (TPS) per *AWS Kinesis* shard. If sustained volume exceeds

this threshold, the system must trigger automated shardsplitting scripts to prevent stream throttling and data loss.

2. Maximum Acceptable Network Latency

- **The Issue:** Deep packet inspection, web application firewalls, and multilayered *API gateways* add processing time to critical financial transactions.
- **The Condition:** The total round-trip latency at the edge interface (*AWS API Gateway* and *AWS WAF*) must remain under 250 milliseconds. If network overhead or cold starts push response times past this boundary, customerfacing transactions will time out and fail over to local backup rules.

3. Maximum Payload Size for Real-Time Streaming

- **The Issue:** Ingesting massive, uncompressed multi-channel logs or bloated third-party market data structures can clog real-time pipelines.
- **The Condition:** Individual message payloads passing through *AWS Kinesis Data Firehose* cannot exceed a hard limit of 1 Megabyte (MB). Documents or files larger than 1MB must be written directly to a staging *AWS S3* bucket, with only the metadata reference sent through the stream.

4. Generative AI Inference Timeout Window

- **The Issue:** Complex *Large Language Model* (LLM) prompts or deep learning risk assessments can hang, delaying transaction completion.
- **The Condition:** The *AWS SageMaker* real-time inference endpoint must return a prediction score or decision within 150 milliseconds. If the model fails to respond within this window, the application circuit breaker trips, bypassing the AI engine to keep core banking transactions processing.

5. Multi-Region Data Drift and Sync Tolerances

- **The Issue:** Cross-region replication over long physical distances faces unavoidable speed-of-light networking delays.
- **The Condition:** The network data drift between the primary production region and the secondary disaster recovery region must not exceed an asynchronous replication lag of 15 seconds for *AWS RDS* and 1 second for *AWS DynamoDB* Global Tables to maintain a valid Recovery Point Objective (RPO).

6. Storage File Fragmentation and Object Limits

- **The Issue:** High-frequency streaming writes a massive number of tiny files to storage, slowing down downstream analytics.
- **The Condition:** *AWS S3* raw data lake partitions must be consolidated whenever the file density drops below an average size of 128 Megabytes (MB) per object. *AWS Glue* compaction jobs must run hourly to merge these fragments and prevent analytical queries from slowing down.

7. Core Database Connection Pool Capacity

- **The Issue:** Sudden, massive waves of serverless database queries can exhaust available engine connections on relational ledgers.
- **The Condition:** The *AWS RDS* relational instance enforces a hard wall at 80% of its maximum concurrent connection pool. Beyond this boundary, serverless backend microservices are forced to queue requests using *AWS RDS Proxy* to prevent database crashes.

8. Unmasked Financial PII Rejection Policies

- **The Issue:** Accidental ingestion of plaintext personal data or unhashed credit card numbers creates immediate regulatory non-compliance.
- **The Condition:** The data pipeline enforces a Zero Plaintext PII policy at the ingestion firewall. If inline *AWS Lambda* regex scanners detect unmasked primary account numbers (PAN) or tax IDs, the entire payload is rejected, quarantined, and flagged for review.

Appendix A: Non-Functional Requirements (NFR) Matrix

This matrix defines the critical quality attributes, operational constraints, and performance benchmarks that the system must satisfy to ensure long-term stability, security, and scalability.

NFR Category	Target Objective	Engineering Validation Mechanism (Tools)	Architecture Component Mapping
Availability	- Ensure 99.99% uptime for user-facing applications and APIs. - Provide multi-AZ redundancy for databases.	- AWS Resilience Hub. - Chaos Engineering (AWS Fault Injection Service)	AWS Route 53, AWS CloudFront, AWS Amplify, AWS API Gateway, AWS RDS
Latency (Query & API)	- API gateway response time < 100ms. - Data lake ad-hoc queries return under 5 seconds.	- AWS CloudWatch Synthetics - AWS X-Ray (Distributed Tracing) - JMeter / Locust	AWS API Gateway, AWS Lambda, AWS Athena, AWS DynamoDB
Data Ingestion Sync Latency	- End-to-end streaming data ingestion latency < 1 minute. - Batch sync completed within scheduled nightly window.	CloudWatch Metrics (IncomingBytes, DeliveryToS3.Latency)	AWS DMS, AWS Kinesis Firehose, AWS AppFlow, AWS Glue
Data Protection	- Achieve 99.999999999% (11 9s) durability for stored data. - Enforce encryption at rest and in transit.	- AWS Config rules - Automated Macie scans	AWS S3 (Raw & Curated), AWS KMS, AWS RDS, AWS Redshift
Perimeter Security	- Block unauthorized network traffic and malicious requests. - Isolate data processing layers from the public internet.	- AWS Network Firewall logs - AWS WAF Security Automations - AWS Inspector	AWS VPC, AWS CloudFront, AWS API Gateway

Identity & Access Management	- Enforce Principle of Least Privilege across all microservices. - Centralize consumer user authentication.	- AWS IAM Access Analyzer - Cognito User Pool Token Validation checks	AWS Cognito, AWS IAM
Throughput Capacity	- Handle up to 10,000 concurrent API requests. - Scale data warehouse queries automatically during peak hours.	- AWS Trusted Advisor - Load Testing tools (BlazeMeter, Artillery)	AWS API Gateway, AWS Lambda, AWS Redshift, AWS DynamoDB
Data Retention	- Automatically transition raw data to cold storage after 90 days. - Retain financial audit records for 7 years.	- S3 Lifecycle Policy simulation tests - AWS Backup compliance reporting	AWS S3 (Raw & Curated), AWS Redshift
System Availability	Maintain high operational availability of entire end-to-end user flows.	CloudWatch ServiceLens dashboards	UI/UX Frontends, AWS Amplify, AWS API

			Gateway, AWS Lambda
Disaster Recovery	- Achieve Recovery Point Objective (RPO) < 15 minutes. - Achieve Recovery Time Objective (RTO) < 1 hour.	- AWS Backup automated restore testing - Disaster Recovery drills	AWS RDS, AWS Redshift, AWS Step Functions
Compute Efficiency	- Optimize resource consumption per execution. - Scale down compute resources to zero when idle.	- AWS Compute Optimizer - Cost Optimization Hub	AWS Lambda, AWS EMR, AWS SageMaker, AWS Step Functions

Storage Optimization	- Convert files to columnar formats (Parquet) to minimize footprint. - Minimize storage costs for analytical queries.	- AWS Glue Schema Registry validation - S3 Storage Lens	AWS S3 (Raw & Curated), AWS Glue
Regulatory Audit	- Track every administrative API action across the cloud environment. - Meet compliance targets (e.g., SOC2, PCI-DSS).	- AWS Audit Manager - CloudTrail Insights	AWS CloudTrail, AWS KMS
Observability Logs	Collect central application logs, model performance data, and system metrics.	- CloudWatch Logs Insights dashboards - SageMaker Model Monitor	AWS CloudWatch, AWS SageMaker (Trained Gen AI Models), AWS SNS
Model Hallucination & Accuracy	- Maintain a faithfulness score above 0.85 (on a 0-1 scale) for RAG responses. - Prevent leakage of toxic or non-compliant text outputs.	- Automated evaluation frameworks (Ragas, TruLens) - AWS Bedrock Guardrails policy simulations	AWS SageMaker (Gen AI Models), AWS Bedrock, Vector Database
Prompt Injection & Input Security	Block 100% of malicious prompt injection attacks, system prompt overrides, and PII leakage attempts.	- Red-teaming automation tools (e.g., garak, Promptfoo) - AWS WAF custom pattern match testing	AWS API Gateway, AWS Bedrock, Guardrails, AWS Lambda
Token Throughput & TTFT (Time to First Token)	- Achieve Time to First Token (TTFT) < 800ms for streaming responses. - Support up to 50 concurrent model invocations without throttling.	- Generative AI load testing tools (LLMperf, Locust custom scripts) - CloudWatch Container Insights	AWS SageMaker Streaming Endpoints, AWS Bedrock, AWS ElastiCache (Semantic Cache)

Appendix B: Cost Optimization & Attribution Matrix

This appendix provides a structured evaluation framework for linking operational expenses to their true business drivers, ensuring financial accountability and resource efficiency.

Cost Domain	Identified Bleed Point	Architectural Fix / Optimization Strategy	Expected Financial Impact
Compute / API Orchestration	OpEx	• Right-size Lambda memory using AWS Lambda Power Tuning tools. • Implement strict API Gateway rate limiting and caching.	• 20% – 40% reduction in serverless compute fees.
Data Lake Storage	OpEx	• Implement S3 Lifecycle Policies to move raw data to Glacier Flexible/Deep Archive after 30 to 90 days. • Enforce Parquet compression via AWS Glue	• 50% – 70% decrease in longterm data storage costs.
Data Ingestion Pipelines	OpEx	• Schedule DMS tasks to run during specific windows using AWS Step Functions. • Tune Firehose buffer sizes/intervals to maximize payload density.	• 15%–30% reduction in streaming ingestion costs.
Big Data Processing & AI	OpEx	• Deploy EMR Managed Scaling and spot instances for transient tasks. • Implement auto-stop lifecycle configurations for SageMaker endpoints.	• 30% – 50% savings on big data analytics and AI modeling workloads.
Relational Database Tier	OpEx	• Convert production RDS instances to AWS RDS • Reserved Instances (RIs). Implement auto-pause for non-production environments using RDS Aurora Serverless v2.	• 30% – 60% savings compared to standard On-Demand pricing.
Enterprise Data Warehouse	OpEx	• Enable Redshift Concurrency Scaling and pause clusters automatically during weekends. • Shift lightweight ad-hoc queries to serverless AWS Athena	• 20% – 35% drop in data warehousing expenditures.

Data Transfer & Networking	OpEx	• Deploy S3 Gateway Endpoints and Interface VPC Endpoints to keep traffic within the AWS private network. • Consolidate workloads into single AZs for non-production environments.	• 40% – 60% elimination of unnecessary data egress and NAT processing fees.
Compute / API Orchestration	OpEx	• Right-size Lambda memory using AWS Lambda Power Tuning tools. • Implement strict API Gateway rate limiting and caching.	• 20% – 40% reduction in serverless compute fees.
Data Lake Storage	OpEx	• Implement S3 Lifecycle Policies to move raw data to Glacier Flexible/Deep Archive after 30 to 90 days. • Enforce Parquet compression via AWS Glue.	• 50% – 70% decrease in longterm data storage costs.
Security, Audit & Monitoring	OpEx	• Set short CloudWatch retention periods (e.g., 7–14 days) and stream older logs to S3 Glacier. • Filter out informational (INFO/DEBUG) logs at the application level before ingestion. • Consolidate multi-region CloudTrail trails into a single organizational trail.	• 25% – 45% reduction in monitoring, ingestion, and security storage overhead.
Model Hosting & Compute	OpEx	• Transition to SageMaker Serverless Inference for unpredictable or low-volume traffic. • Use auto-scaling policies to scale instance counts to zero or minimum levels during offpeak windows.	• 40% – 60% reduction in continuous compute infrastructure spend.
Gen AI Model Inferences	OpEx	• Implement Semantic Caching (e.g., using Redis/DynamoDB) to intercept identical queries. • Deploy a Model Routing routing layer to send basic queries to lighter models and reserve heavy models for complex reasoning.	• 30% – 50% drop in third-party or native token costs.
Vector DB & RAG Storage	OpEx	• Apply strict data chunking filtering before embedding generation to avoid indexing noise.	• 20% – 35% savings

		• Implement a lifecycle policy inside the Vector Database to purge or archive stale historical embeddings.	
--	--	--	--

Appendix C: Enterprise Security Risk Register

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

Security Vulnerability Description	Likelihood (1-5)	Impact (1-5)	Risk Rating (1-25)	Core Strategy	Technical	Mitigation
Generative AI Prompt Injection & Jailbreaking: Attackers manipulate LLM input tokens to bypass system guardrails, extract internal configurations, or generate toxic/restricted responses.	4	4	16 (High)	Implement AWS Bedrock Guardrails to screen user inputs. Deploy input sanitization libraries on the API Gateway layer. Treat output text as untrusted data.		
Partner Identity Spoofing / Account Takeover: Malicious actors compromise an external hospital or broker IdP account to impersonate authorized personnel.	3	5	15 (High)	Enforce strict session token expiration (15–60 mins) and deploy multi-factor authentication (MFA) step-up actions via AWS IAM Identity Center.		
Accidental PHI Leakage in Unstructured Fields: Partner applications or users accidentally upload un-redacted clinical notes or PDFs containing raw patient names or SSNs.	4	4	16 (High)	Trigger an automated serverless workflow using AWS Macie and AWS Lambda to isolate, sanitize, and quarantine matching files in the Raw zone.		
Unencrypted Data Transit & Data Lake Leakage: Sensitive enterprise data or PII moving between ingestion layers (DMS, Kinesis) and S3 bucket tiers is intercepted or exposed publicly due to misconfiguration.	3	5	10 (Medium)	Enforce S3 Bucket Policies with explicit aws:SecureTransport Deny rules. Enable AWS KMS default encryption using Customer Managed Keys (CMK).		
API Gateway Broken Object Level Authorization (BOLA): Exposed endpoints allow an authenticated user to access or manipulate data parameters belonging to another tenant or enterprise client.	3	4	12 (Medium)	Validate tenant ownership parameters within AWS Lambda Authorizers. Implement claimsbased verification using custom tokens issued by AWS Cognito.		

Vector Database Poisoning & Embedding Manipulation: An attacker introduces malicious document chunks into the enterprise data lake, corrupting RAG context retrieval and forcing faulty model logic.	2	4	8 (Low)	Implement mandatory multi-stage validation inside AWS Glue data pipelines. Require cryptographic signing (SHA-256) for all files entering the raw S3 ingestion zone.
Insecure Secrets & Hardcoded ThirdParty API Keys: Developers hardcode access tokens for proprietary GenAI models, vector indices, or relational DB systems into code repositories or AWS Lambda environments.	3	4	12 (Medium)	Force migration environment strings Secrets Manager. automatic secret Implement IAM execu instead of long-lived access keys.
Distributed Denial of Wallet (DDoW) via Serverless Resource Exhaustion: Automated bot traffic targets publicfacing API Gateway routes, triggering uncapped AWS Lambda scaling loops and model token consumption, causing massive unexpected operational cost spikes.	3	3	9 (Low)	Configure strict API Gateway Usage Plans and Throttling rate limits. Deploy AWS WAF with ratelimiting rulesets and Web ACL blocklists to drop burst attacks.
Model Inversion & Training Data Extraction: Malicious actors craft specialized prompt queries to trick the model into outputting exact training data, exposing proprietary code or hidden PII.	3	4	12 (Medium)	Implement strict PII redaction pipelines using AWS Macie before data hits the finetuning/RAG layer. Apply regex filters in AWS Bedrock Guardrails to block downstream outputs containing phone numbers, emails, or system IDs.
Log Leakage of Sensitive AI Interactions: User prompts containing highly confidential trade secrets or customer PII are logged in plain text into CloudWatch or SageMaker Model Monitor, violating compliance rules.	4	3	12 (Medium)	Enforce CloudWatch Logs data protection policies to automatically mask PII. Encrypt all log groups with a dedicated AWS KMS Customer Managed Key instead of the default AWSmanaged key.

AI Software Supply Chain & Dependency Poisoning: Vulnerabilities or malicious code hidden within open-source GenAI frameworks (e.g., specific versions of LangChain, LlamaIndex, or Hugging Face libraries) compromise the AWS Lambda or EMR execution environments.	3	4	12 (Medium)	Stream all AWS CloudTrail and IBM System Management Facilities (SMF) logs continuously to an immutable, write-once-read-many (WORM) centralized log storage engine.
Indirect Prompt Injection via Ingested Corporate Data: An attacker uploads a malicious file containing hidden prompt instructions into the raw data lake tier. When processed and retrieved by RAG, it hijacks the LLM session logic downstream.	3	4	12 (Medium)	Implement structural parsing and hygiene checks inside AWS Glue. Sanitize all incoming file text using an independent LLM classification guardrail layer to flag formatting that mimics system prompt directives before storage.

Appendix D: Enterprise RACI Matrix

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

Key Architectural Deliverable	Cloud Infra Team	Data Engineering Team	Gen AI/ML Team	Cyber Security (SecOps)	Business Team	FinOps & Finance
Infrastructure Provisioning & MultiAZ Availability	A/R	C	C	C	I	I
Ingestion Pipeline Management & Sync Latency`	C	A/R	C	I	I	I
Model Hosting, Tuning & Semantic Cache Layer	C	C	A/R	C	I	I
Perimeter Security, WAF Rules & IAM Guardrails	R	I	I	A	I	I
Data Protection, Encryption Keys & PII Redaction	R	R	I	A	C	I
GenAI Vulnerability Assessment (Prompt Injection / Hallucinations)	I	I	R	A	C	I
Storage Lifecycle Policies & Compute Right-Sizing	R	R	R	I	I	A
Observability, Incident Response & Central Log Ingestion	A/R	R	R	R	I	I
Disaster Recovery Strategy & Automated RPO/RTO Testing	A/R	R	C	C	C	I

About the Author

Amit Kulkarni

Senior Solutions Architect

Pune, India | amitkwork2920@gmail.com | [LinkedIn](#) | [Github](#)

I'm a Technology-agnostic architecture leader having 18+ years of partnership with 15+ clients across BFSI, Retail, Insurance, Healthcare, Travel, Media, and the Public Sector. Proven track record in cloud transformation (AWS, Azure), legacy COTS and mainframe modernization, security posture assessments, microservices modernization, event-driven architectures, and working knowledge on AI initiatives.

Architectural Specialization (Recent Focus)

During a recent deliberate transition period away from formal corporate employment, I dedicated my time to advanced deep-dives into enterprise cloud topologies, edge cases, and failure domains. This case study on *GenAI based Solution Architecture for Banking and Capital Markets on AWS* is part of a broader body of architectural research aimed at critiquing standard vendor blueprints, identifying hidden system vulnerabilities, and building productionready engineering remediations for high-throughput transactional environments.

I am currently seeking my next high-impact engineering role where I can solve complex distributed systems challenges and help scale robust, mission-critical infrastructure teams.